

RAJALAKSHMI INSTITUTE OF TECHNOLOGY

KUTHAMBAKKAM, CHENNAI - 600 124



rit

RAJALAKSHMI
INSTITUTE OF
TECHNOLOGY

BELIEVE IN THE POSSIBILITIES
AN AUTONOMOUS INSTITUTION

Laboratory Record Note Book

NAME:

SUBJECT:

BRANCH:

REG. No:

ROLL No:

SEMESTER:

ACADEMIC YEAR:

RAJALAKSHMI INSTITUTE OF TECHNOLOGY

KUTHAMBAKKAM, CHENNAI - 600 124

BONAFIDE CERTIFICATE

NAME

ACADEMIC YEAR.....YEAR..... BRANCH

REGISTER No.

Certified that this is the bonafide record of work done by the above student in the

.....Laboratory during the year **20** - **20**

Signature
Faculty - in - Charge

Signature
H.O.D

Submitted for the Practical Examination held on.....

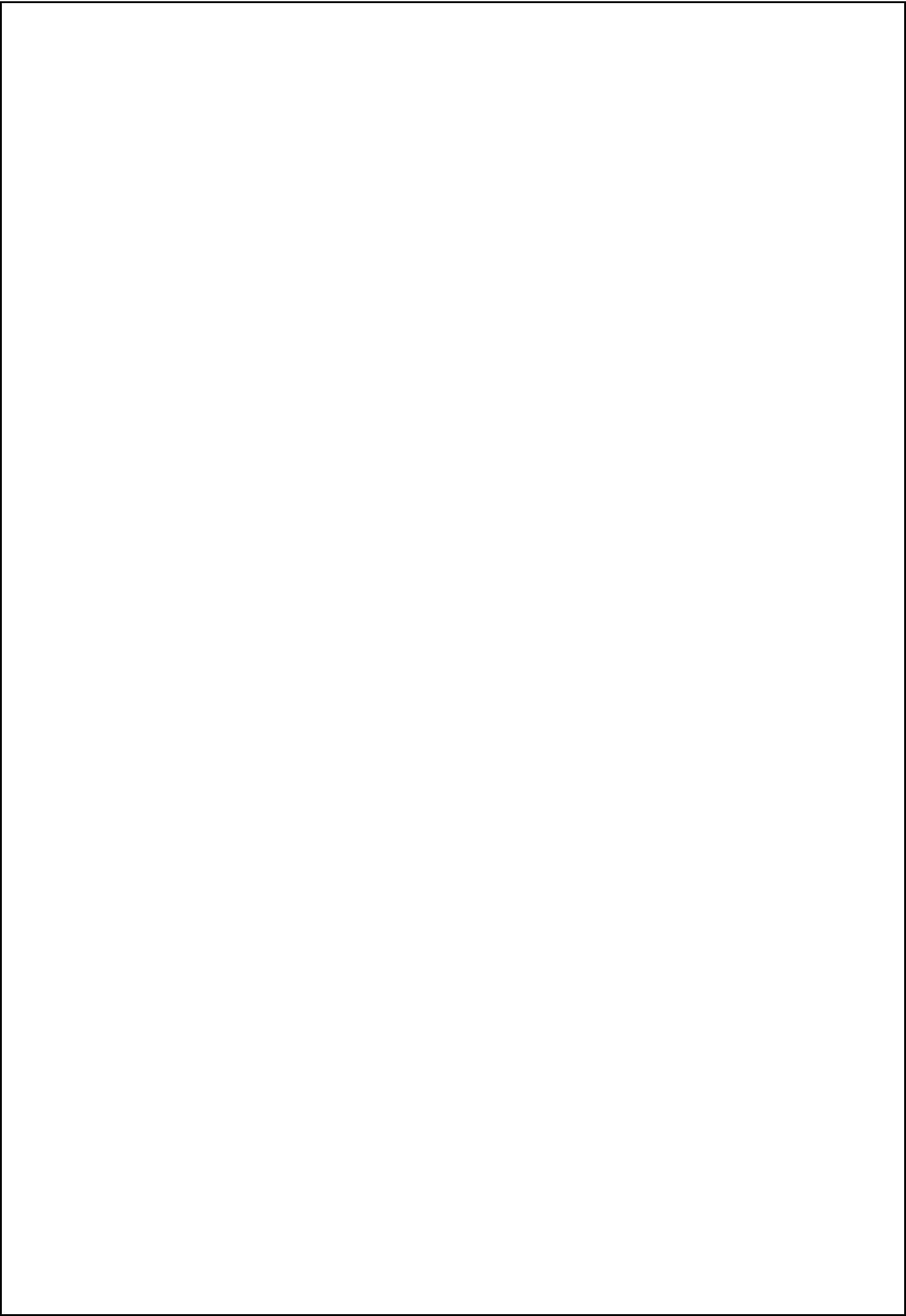
Internal Examiner

External Examiner

INDEX

Name: _____ **Branch:** _____ **Sec:** _____ **Reg No.:** _____

[illegible]



EX. NO: 1(a)	IMPLEMENTATION OF CAESAR CIPHER
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
text = "HELLO"
key = 3
enc = "".join([chr((ord(c)-65+key)%26+65) for c in text]) #Encrypt
dec = "".join([chr((ord(c)-65-key)%26+65) for c in enc]) #Decrypt
print("Encrypted:", enc)
print("Decrypted:", dec)
```

OUTPUT:

Encrypted: KHOOR

Decrypted: HELLO

RESULT:

EX. NO: 1(b)	IMPLEMENTATION OF VIGNERE CIPHER
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```

text, key = "HELLO", "KEY"
key = (key * (len(text)//len(key)+1))[:len(text)]
enc = "".join([chr((ord(t)+ord(k)-2*65)%26+65) for t,k in zip(text,key)])
dec = "".join([chr((ord(e)-ord(k)+26)%26+65) for e,k in zip(enc,key)])
print("Encrypted:", enc)
print("Decrypted:", dec)

```

OUTPUT:

Encrypted: RIJVS

Decrypted: HELLO

RESULT:

EX. NO: 1(C)	IMPLEMENTATION OF PLAYFAIR CIPHER
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
from pycipher import Playfair
pf = Playfair('KEYWORD') # Create cipher with
keyword enc = pf.encrypt('HELXLO') # X used for
repeating letters dec = pf.decrypt(enc)
print("Encrypted:", enc)
print("Decrypted:", dec)
```

OUTPUT:

```
Encrypted: DMRCXG
Decrypted: HELXLO
```

RESULT:

EX. NO: 2	IMPLEMENTATION OF DES AND AES ALGORITHMS
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```

from Crypto.Cipher import DES, AES
from Crypto.Random import get_random_bytes

# --- DES ---
des_key = get_random_bytes(8) # DES key (8
bytes) des = DES.new(des_key,
DES.MODE_ECB)
msg = b"8ByteMsg" # Must be 8
bytes cipher_des =
des.encrypt(msg) plain_des =
des.decrypt(cipher_des)
# --- AES ---
aes_key = get_random_bytes(32) # AES-256
key aes = AES.new(aes_key,
AES.MODE_ECB) msg2 =
b"16ByteMessage!!" # Must be 16 bytes
cipher_aes = aes.encrypt(msg2)
plain_aes = aes.decrypt(cipher_aes) print("DES Decrypted:", plain_des) print("AES
Decrypted:", plain_aes)

```

OUTPUT:

DES Decrypted: b'8ByteMsg'

AES Decrypted: b'16ByteMessage!!'

RESULT:

EX. NO: 3	IMPLEMENTATION OF RSA ALGORITHM
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```

p, q = 13, 17
n = p * q
phi = (p-1)*(q-1)
e = 5
d = pow(e, -1, phi) # Modular
inverse msg = 10
cipher = pow(msg, e, n) #Encrypt
plain = pow(cipher, d, n) # Decrypt
print("Decrypted:", plain)

```

OUTPUT:

Decrypted: 10

RESULT:

EX. NO: 4	IMPLEMENTATION OF MD5 AND SHA-256 ALGORITHMS
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
import hashlib
text = "hello world"
md5_hash = hashlib.md5(text.encode()).hexdigest()
sha256_hash =
hashlib.sha256(text.encode()).hexdigest() print("Input
:", text)
print("MD5 Hash :", md5_hash) print("SHA-256 Hash:", sha256_hash)
```

OUTPUT:

```
Input : hello world
MD5 Hash : 5eb63bbbe01eeed093cb22bb8f5acdc3
SHA-256 Hash:
b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9
```

RESULT:

EX. NO: 5	IMPLEMENTATION AND VERIFICATION OF DIGITAL SIGNATURE
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives import serialization
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
)
public_key = private_key.public_key()
message = b"Hello, this is a digital signature test."
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
print("Signature generated successfully.\n")
try:
    public_key.verify(
        signature,
        message,

```

```
padding.PSS(  
    mgf=padding.MGF1(hashes.SHA256()),  
    salt_length=padding.PSS.MAX_LENGTH  
)  
hashes.SHA256()  
)  
print("Signature Verified: VALID")  
except Exception as e:  
    print("Signature Verification Failed:", str(e))
```

OUTPUT:

Signature generated successfully.
Signature Verified: VALID

RESULT:

EX. NO: 6	SETTING UP AND CONFIGURING SSL ON A WEB SERVER
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
from flask import Flask
app=Flask(__name__)
@app.route('/')
def home():
    return "This Flask app is running with HTTPS (SSL Enabled)!"
if __name__=="__main__":
    app.run(host='0.0.0.0',port=443,ssl_context=('cert.pem','key.pem'))
```

OUTPUT:

This Flask app is running with HTTPS (SSL Enabled)!

RESULT:

EX. NO: 7	INSTALLATION AAND CONFIGURATION OF SNORT IDS
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```

from scapy.all import sniff,IP,TCP
from datetime import datetime
blacklist_ips=['192.168.1.100','10.0.0.5']
blacklist_ports=[4444,5555]
def packet_callback(packet):
    if IP in packet:
        src_ip=packet[IP].src
        dst_ip=packet[IP].dst
        if TCP in packet:
            sport=packet[TCP].sport
            dport=packet[TCP].dport
            timestamp=datetime.now().strftime('%Y-%m-%d %H:%M:%S')
            if src_ip in blacklist_ips or dst_ip in blacklist_ips:
                print(f"[ALERT] [{timestamp}] Suspicious IP detected: {src_ip} -> {dst_ip}")
            if sport in blacklist_ports or dport in blacklist_ports:
                print(f"[ALERT] [{timestamp}] Suspicious port usage: {src_ip}:{sport} -> {dst_ip}:{dport}")
print("Starting IDS... Press Ctrl+C to stop.")

```

```
sniff(prn=packet_callback,store=0)
```

OUTPUT:

Starting IDS... Press Ctrl+C to stop.

[ALERT] [2025-05-30 15:20:12] Suspicious IP detected: 192.168.1.100 -> 192.168.1.10

[ALERT] [2025-05-30 15:20:13] Suspicious port usage: 10.0.0.2:4444 -> 192.168.1.10:80

RESULT:

EX. NO: 8	SETTING UP AND CONFIGURING A FIREWALL USING IPTABLES
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
sudo iptables -F
sudo iptables -X
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW,ESTABLISHED -j ACCEPT
sudo iptables -A OUTPUT -p tcp --sport 22 -m state --state ESTABLISHED -j ACCEPT
sudo iptables -A INPUT -p tcp -m multiport --dports 80,443 -m state --state
NEW,ESTABLISHED -j ACCEPT
sudo iptables -A OUTPUT -p tcp -m multiport --sports 80,443 -m state --state
ESTABLISHED -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables-save | sudo tee /etc/iptables/rules.v4 > /dev/null
```

OUTPUT:

```
sudo iptables -L -n -v
```

RESULT:

EX. NO: 9	TESTING AND MITIGATING SQL INJECTION ATTACKS
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
import sqlite3
conn=sqlite3.connect(':memory:')
cursor=conn.cursor()
cursor.execute("CREATE TABLE users (username TEXT, password TEXT)")
cursor.execute("INSERT INTO users VALUES (?,?)",('admin','admin123'))
user_input="" OR '1'='1"
query="SELECT * FROM users WHERE username=? AND password=?"
cursor.execute(query,(user_input,user_input))
print("Logged in!" if cursor.fetchone() else "Login failed.")
```

OUTPUT:

Login failed — SQL injection no longer works as inputs are treated as plain data.

RESULT:

EX. NO: 10	IDENTIFYING AND MITIGATING XSS VULNERABILITIES
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
import html
user_input="<script>alert('XSS')</script>"
print("Raw User Input (Unsafe):",user_input)
safe_output=html.escape(user_input)
print("Sanitized Output (Safe for Rendering):",safe_output)
html_template=f"<html><body><p>User said: {safe_output}</p></body></html>"
print("\n--- Simulated Rendered HTML Page ---")
print(html_template)
```

OUTPUT:

```
Raw User Input (Unsafe): <script>alert('XSS')</script>
Sanitized Output (Safe for Rendering):
<script>&lt;script&gt;alert(&#x27;XSS&#x27;)&lt;/script&gt;
--- Simulated Rendered HTML Page ---
<html><body><p>User said:
&lt;script&gt;alert(&#x27;XSS&#x27;)&lt;/script&gt;</p></body></html>
```

RESULT:

EX. NO: 11	IMPLEMENTING INPUT VALIDATION AND ERROR HANDLING
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
try:
    age=int(input("Enter your age: "))
    if age<0:
        raise ValueError("Age cannot be negative.")
    print("Your age is:",age)
except ValueError as e:
    print("Error:",e)
```

OUTPUT:

```
Enter your age: 20
Your age is: 20
Enter your age: -5
Error: Age cannot be negative.
Enter your age: abc
Error: invalid literal for int() with base 10: 'abc'
```

RESULT:

EX. NO: 12	IMPLEMENTING ENCRYPTION IN DATA STORAGE
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```

from cryptography.fernet import Fernet
import sqlite3
key=Fernet.generate_key()
fernet=Fernet(key)
conn=sqlite3.connect(':memory:')
cursor=conn.cursor()
cursor.execute("CREATE TABLE users (name TEXT,password TEXT)")
plain_password="my_secret_pwd"
enc_password=fernet.encrypt(plain_password.encode()).decode()
cursor.execute("INSERT INTO users VALUES (?,?)",("admin",enc_password))
cursor.execute("SELECT password FROM users WHERE name=?",("admin",))
enc_pwd_from_db=cursor.fetchone()[0]
dec_password=fernet.decrypt(enc_pwd_from_db.encode()).decode()
print("Decrypted Password:",dec_password)

```

OUTPUT:

Decrypted Password: my_secret_pwd

RESULT:

EX. NO: 13	CONFIGURING A VPN FOR SECURE COMMUNICATION
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
import socket
import threading
import time
def xor_encrypt_decrypt(data,key):
    return bytes([b^key for b in data])
key=123
def run_server():
    server_socket=socket.socket()
    server_socket.bind(('localhost',5000))
    server_socket.listen(1)
    print("[Server] Listening on port 5000...")
    conn,addr=server_socket.accept()
    print(f"[Server] Connected to: {addr}")
    encrypted_data=conn.recv(1024)
    decrypted_data=xor_encrypt_decrypt(encrypted_data,key)
    print("[Server] Decrypted message from client:",decrypted_data.decode())
    response="Hello Secure Client!".encode()
    encrypted_response=xor_encrypt_decrypt(response,key)
    conn.send(encrypted_response)
```

```
conn.close()
server_socket.close()
def run_client():
    time.sleep(1)
    client_socket=socket.socket()
    client_socket.connect(('localhost',5000))
    print("[Client] Connected to server.")
    message="Hello Secure Server!".encode()
    encrypted_message=xor_encrypt_decrypt(message,key)
    client_socket.send(encrypted_message)
    encrypted_reply=client_socket.recv(1024)
    reply=xor_encrypt_decrypt(encrypted_reply,key)
    print("[Client] Decrypted message from server:",reply.decode())
    client_socket.close()
if __name__=="__main__":
    server_thread=threading.Thread(target=run_server)
    server_thread.start()
    run_client()
    server_thread.join()
```

OUTPUT:

```
[Server] Listening on port 5000...
[Client] Connected to server.
[Server] Connected to: ('127.0.0.1', XXXXX)
[Server] Decrypted message from client: Hello Secure Server!
[Client] Decrypted message from server: Hello Secure Client!
```

RESULT:

EX. NO: 14	CONDUCTING A RISK ASSESSMENT FOR A GIVEN SCENARIO
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```

risk_data=[
{"asset":"User Credentials","threat":"SQL Injection","vulnerability":"Unvalidated Input","likelihood":"High","impact":"High"},
{"asset":"Sensitive Data","threat":"Data Theft","vulnerability":"Weak Access Controls","likelihood":"Medium","impact":"High"},
{"asset":"Server","threat":"DoS Attack","vulnerability":"Open Ports","likelihood":"Low","impact":"Medium"}
]
risk_matrix={
("High","High"):"Critical",
("High","Medium"):"High",
("High","Low"):"Moderate",
("Medium","High"):"High",
("Medium","Medium"):"Moderate",
("Medium","Low"):"Low",
("Low","High"):"Moderate",
("Low","Medium"):"Low",
("Low","Low"):"Low"
}
mitigations={

```

```

"Unvalidated Input":"Use parameterized queries",
"Weak Access Controls":"Implement RBAC",
"Open Ports":"Use firewall and restrict unused ports"
}
header=f"{'Asset':<20} {'Threat':<20} {'Risk Level':<15} {'Mitigation'}"
separator="-"*len(header)
print(header)
print(separator)
for item in risk_data:
    key=(item["likelihood"],item["impact"])
    risk_level=risk_matrix.get(key,"Unknown")
    mitigation=mitigations.get(item["vulnerability"],"Apply security best practices")
    print(f"{'item['asset']':<20} {'item['threat']':<20} {'risk_level':<15} {'mitigation'}")

```

OUTPUT:

Asset	Threat	Risk Level	Mitigation

User Credentials	SQL Injection	Critical	Use parameterized queries
Sensitive Data	Data Theft	High	Implement RBAC
Server	DoS Attack	Low	Use firewall and restrict unused ports

RESULT:

EX. NO: 15	PERFORMING INCIDENT RESPONSE FOR A SIMULATED ATTACK
DATE:	

AIM:

ALGORITHM:

PROGRAM:

```
log=["login failed","login failed","<script>alert('xss')</script>","login failed"]
alert=False
for entry in log:
    if "<script>" in entry or "failed" in entry:
        alert=True
if alert:
    print("[ALERT] Suspicious activity detected.")
    print("1. User disabled.\n2. Input filtering applied.\n3. Logs archived.")
    print("4. System recovered.\n5. Incident documented.")
```

OUTPUT:

```
[ALERT] Suspicious activity detected.
1. User disabled.
2. Input filtering applied.
3. Logs archived.
4. System recovered.
5. Incident documented.
```

RESULT: